SOURCE-PINNED TECHNICAL REVIEW

Asymptotic

An incremental correctness audit

Affine Lerch expansions, omitted constants, and the difference between a cutoff and a remainder grade

Prepared for Vladimir Reshetnikov
10 September 2026

Reviewed repository

https://github.com/VladimirReshetnikov/Asymptotic **Pinned revision**

`8f280847bf8fd1f6488834cadf1542867292ce10`

Principal result. One newly substantiated soundness regression is isolated in the recently integrated affine Lerch constructor. A discarded constant is charged to a tail that can decay to zero. The article proves the failure, provides a narrowly scoped candidate repair, and supplies independent exact arithmetic tests and unexecuted kernel regressions.

Execution boundary. No Wolfram or Mathics package execution succeeded in this review environment. Public outputs below are source-derived predictions, not runtime transcripts. The executed evidence comprises 20 independent Python test methods, four synthetic patch-emitter fixture methods, and a separate symbolic identity check. There is no full-suite or patched-package acceptance claim.

Abstract

The repository already retains 45 review packages across six waves. Repeating that corpus's correctness and development backlog would not satisfy the present request. This review instead examines selected canonical mathematical and compatibility modules, uses the maintained review record as an exclusion baseline, and follows a recently added implementation through its proof obligations.

The resulting new finding concerns affine Lerch expansions at nonpositive exclusive cutoffs. For example, the current source predicts a zero retained expression and an absolute remainder bound $3/x^2$ for $1 + \Phi(1/2, 2, x)$ at cutoff zero. At $x = 2$, the defining series proves that the actual error is at least $5/4$, whereas the predicted bound is $3/4$. Moreover, the error tends to one, so the advertised $\mathcal{O}(x^{-2})$ statement is false, not merely a loose quantitative estimate.

The cause is a confusion between the requested retention cutoff and the first omitted atom exponent. The correct common remainder grade is the minimum of the atom grade and zero whenever a nonzero constant is omitted. A proof, metadata repair, boundary matrix, source-guarded patch emitter, and exact rational oracle are supplied. A finite-source edge case is treated separately, with its public reachability explicitly left unestablished.

Contents

1 Executive assessment **2**

2 Snapshot, review coverage, and evidential limits **2**

3 Nonduplication and the implementation boundary **4**

4 The contracts that the new path must preserve **5**

5 N01: exact source trace and counterexample **5**

6 A correct remainder join **7**

7 The finite-source edge case **8**

8 Candidate source repair **8**

9 Independent tests and reproduction artifacts **9**

10 Wolfram 15 and Mathics3 assessment for this finding **11**

11 Performance, API, and local development recommendations **11**

12 Conclusion **12**

A Archive contents and commands **12**

1 Executive assessment

1.1 The new finding

N01 — an omitted affine constant can inherit a falsely decaying Lerch remainder. This is a mathematical soundness issue in a shared kernel module, not a numerical accuracy issue and not specifically a Mathics implementation problem. Its impact includes the asymptotic remainder order, the explicit absolute bound, and the meaning of the displayed frontier term. The relevant implementation is `dirichletLerchForward` in `DirichletSpecialFunctions.wl`. The affine recognition and constant-retention helpers are in the same file. [1]

The issue deserves a high correctness priority because the constructor does not merely reject an input or return a less precise approximation: its advertised bound is contradicted by an exact positive summand. Nevertheless, this review does not label an unexecuted public prediction as a reproduced kernel failure. The distinction matters on both requested runtimes.

1.2 What to change

The candidate repair leaves the exclusive cutoff and all retained terms unchanged. It treats an omitted constant as an error contribution of exponent zero, joins that exponent with the atom's tail exponent, recomputes the common bound, and updates the frontier metadata consistently. It does not broaden function admission, alter term-goal policy, or replace the moment algorithm.

For the main witness, a repaired zero approximation has a sound common contract $0 + \mathcal{O}(1)$, with the conservative absolute bound 3 on $x \geq 1$. A caller seeking a convergent approximation should instead retain the constant: at positive cutoff one, the intended result is $1 + \mathcal{O}(x^{-2})$. The latter is a source-derived control and an unrun kernel test, not an independently observed package output.

1.3 Why the article has one numbered defect

Several plausible issues in the inspected areas were already recorded in the repository's review register or wave indexes. They are excluded rather than renumbered. The positive-cutoff tests of the new affine constructor do not exercise the N01 boundary. The single root cause is therefore reported once, with several consequences and edge cases, rather than split into artificial findings for each affected field.[3, 4, 5, 6, 8]

2 Snapshot, review coverage, and evidential limits

2.1 Source pinning

All current-source conclusions in this article refer to revision

`8f280847bf8fd1f6488834cadf1542867292ce10`.

Earlier reads encountered another revision; the material used for the final finding was re-read at the pin above. Immutable source links, not references to an unspecified moving `main`, are used throughout. The review date is 10 September 2026.

The canonical modular source, rather than a reconstructed standalone, is the basis of the code reasoning. A complete local checkout was not obtained. The GitHub connector returned selected source files

and ranges; its responses were sufficient for the concrete control-flow analysis, but not for building or executing the whole distribution.

2.2 What was inspected

The following table records the substantive current-source review used in this report. “Full” refers to the returned text of that module being inspected, not to execution or formal verification.

Module or area	Depth	Focus
`AsymptoticAnalysis.wl`	Selected ranges	Exact-number admission, coefficient normalization, sparse jets, forward assembly, inverse construction, coefficient queries, residuals and perturbative inversion.
`DirichletSpecialFunctions.wl`	Full	Affine admission, cutoff selection, geometric moments, explicit bounds and result assembly.
`SeriesArithmetic.wl`	Selected range	Held normalization, regular operands, arithmetic dispatch and precision use.
`InverseCertificates.wl`	Selected ranges	Exact interval primitives, branch-domain checks, residual root enclosures and driver accuracy handling.
`BarnesForward.wl`	Full	Shift identities and Bernoulli-tail construction.
`ParameterizedSpecialFunctions.wl`	Full	Argument-position admission and Frobenius reductions.
`MathicsAssumptions.wl`	Full	Realness and sign proof rules.
`MathicsCompatibility.wl`	Full	Scoped evaluator and association shims.
`MathicsTaylor.wl`	Full	Restricted defining-series Taylor fallback.
`MathicsNumerical.wl`	Full	Numerical-reference precision guard and logarithm fallback.
`MathicsLists.wl`	Full	Empty-list mapping adapter and installation scope.
`ReviewWave6Boundaries.wlt`	Full	Existing focused tests of the newly integrated boundaries.

This is targeted repository analysis, not a line-by-line certification of every module, test, build script, document, or historical report. No conclusion of “no remaining defects” follows for an area with no new numbered finding. The machine-readable coverage record is `evidence/source-coverage.json`.

2.3 Runtime and tool limitations

Wolfram service calls did not yield an executable session, a local Wolfram kernel was unavailable, and Mathics could not be installed after network resolution failed. No package-level performance

timing, baseline transcript, patched-kernel result, or cross-runtime differential result is therefore claimed. Existing upstream acceptance records remain upstream evidence only; they are not combined with this review's Python counts.

The independent oracle uses exact rational arithmetic and the convergent Lerch defining sum. It does not translate Wolfram evaluation semantics. The candidate emitter was tested against synthetic text anchors, not against a complete materialized checkout. These limitations are recorded in the archive as data as well as prose.

3 Nonduplication and the implementation boundary

3.1 The nearest earlier report

Wave-6 report 55 identified the inability of the older constructor to accept a simple affine wrapper around a defining-sum atom. Its snapshot was `8cee870994f506b501bae3ea6bd4a3a7edb895c1`. Its supplied prototype lifted the atom through existing arithmetic and then truncation. It explicitly characterized the original problem as a coverage failure, not a false remainder theorem.[7]

The current source now includes `dirichletSpecialAffineForm` and `dirichletSpecialAffineConstant`. Its comments attribute that integration to report 55 N01. The present finding is in this new direct integration's handling of an omitted constant, after affine admission has succeeded. It is not a recommendation to add affine support again.[1]

3.2 What is deliberately not reported again

Report 55 also discusses term goals and warns that affine cancellation must be accounted for when counting final blocks. That subject is excluded here. Likewise, the existing register and later wave indexes already identify broad precision, source-admission, arithmetic, compatibility, certification and resource topics in the inspected modules. Those existing recommendations are not restated as this review's roadmap.[4, 5, 6, 7]

The novelty claim is consequently narrow: the current direct affine Lerch constructor uses the unchanged atom tail grade when charging a discarded constant. The previous report's admission defect and the current branch's soundness defect require different repairs and have different witnesses. The attached novelty ledger records this distinction. It is not a claim that a semantic absence theorem has been proved over every historical document.

3.3 A relevant gap in the new tests

The affine test in `ReviewWave6Boundaries.wlt` checks Zeta translations, negative multipliers, one positive-cutoff Lerch translation, bound comparisons, and term-goal behavior. Its Lerch example uses cutoff three. None of those Lerch cases omits a nonzero constant at a nonpositive cutoff. The existing positive-cutoff control can therefore pass while N01 remains. This observation is specific to the inspected test file; it does not assert that no other test anywhere in the repository mentions cutoff zero. [8]

4 The contracts that the new path must preserve

4.1 Coordinates, retained terms, and the first omitted term

Write $a = A(x) \to +\infty$ and $w = 1/a \to 0^+$. In the Lerch adapter, a retained row (p, c) means $c\,w^p = c\,a^{-p}$. A cutoff h is exclusive: rows satisfy $p < h$. By contrast, ρ is the exponent of the first omitted nonzero atom coefficient, or $+\infty$ when the atom expansion is exact and exhausted. These two numbers serve different purposes.[1]

For a sparse expansion, ρ can exceed h by a large gap. In particular, $h \le 0$ does not imply $\rho \le 0$. This simple logical distinction is the entire source of the failure: the omission test for a constant uses h, while the quantitative charging formula assumes a sign restriction on ρ that was never established.

4.2 The analytic object

On the positive real a-axis and $|z| < 1$,

$$\Phi(z, s, a) = \sum_{n=0}^{\infty} \frac{z^n}{(a+n)^s}. \tag{1}$$

This is the defining-sum regime relevant here. No continuation across a cut, negative a, regularization at a pole, or ambiguous complex power is needed for the witness.[9, 10]

For fixed real s, the adapter writes a large-a expansion with exponents $s + k$ and coefficients

$$c_k = \frac{(-1)^k (s)_k}{k!} M_k(z), \qquad M_0(z) = \frac{1}{1-z}, \quad M_k(z) = \sum_{n=0}^{\infty} n^k z^n \quad (k > 0). \tag{2}$$

The implementation uses the corresponding polylogarithmic moments. The new finding does not challenge that moment formula.[1]

The affine source is $F(a) = \alpha\Phi(z, s, a) + \beta$, with fixed real coefficients. A nonzero constant contributes weight zero. When it is not retained, it remains part of the error even when all of the atom's errors have strictly positive weights.

5 N01: exact source trace and counterexample

5.1 Public request to characterize

The supplied kernel probe includes the following request, with the package backend explicit so that a native backend does not mask the path:

```
s = AsymptoticExpansion[
  1 + LerchPhi[1/2, 2, x], {x, Infinity, 0},
  "Backend" -> "Package"];
{Normal[s], s["RemainderPower"],
 s["AbsoluteRemainderBound"], s["FrontierTerm"]}
```

The source-derived prediction is

$$\{0,\ 2,\ 3/x^2,\ 2/x^2\}. \tag{3}$$

Equation (3) is *not* a captured Wolfram or Mathics output. The kernel probe must still verify it in each runtime.

5.2 Control-flow derivation

The affine recognizer identifies the atom with $\alpha = 1$, $\beta = 1$. At $s = 2$, the first coefficient is $c_0 = M_0(1/2) = 2$, of exponent two. Because $2 \not\leq 0$, the moment-selection loop retains no row and sets its frontier to $(2, 2, 0)$. Thus the atom's remainder grade is $\rho = 2$.

The constant helper sees a nonpositive cutoff and returns the constant as `charged`, rather than adding a retained weight-zero row. That choice is consistent with the exclusive cutoff. The error occurs in the next step: the code adds $|\beta|$ to the atom's bound constant but continues to multiply by $a^{-\rho}$, leaving ρ unchanged.[1]

```
If[charged =!= 0,
  boundConstant = boundConstant + Abs[charged];
  bound = boundConstant argument^(-rho)];
```

For $N = 0$, the atom's bound constant is 2. The operation above changes it to 3 and claims the full omitted source is bounded by $3/x^2$. The subsequent result assembly receives the same $\rho = 2$ and empty retained rows. This propagates the mistake into the asymptotic object rather than confining it to an auxiliary diagnostic.[1]

5.3 A finite exact contradiction

At $x = 2$, the first summand in (1) is $1/4$, and all summands are positive. Hence

$$1 + \Phi(1/2, 2, 2) \;\geq\; 1 + \frac{1}{4} = \frac{5}{4} \;>\; \frac{3}{4} = \frac{3}{x^2}\bigg|_{x=2}. \tag{4}$$

The predicted retained expression is zero, so this is directly a contradiction of the predicted absolute error bound. The point $x = 2$ satisfies both the positive-argument domain and the bound's $a \geq 1$ condition. No numerical special-function evaluator is involved.

5.4 An asymptotic contradiction, not just a bad finite constant

For $a > 0$,

$$\frac{1}{a^2} \leq \Phi(1/2, 2, a) \leq \frac{1}{a^2} \sum_{n=0}^{\infty} 2^{-n} = \frac{2}{a^2}. \tag{5}$$

It follows that $1 + \Phi(1/2, 2, a) \to 1$. Therefore this function is not $\mathcal{O}(a^{-2})$. In fact, its ratio to the claimed scale is at least a^2 and diverges. Increasing an internal Taylor order, floating-point precision, or resource budget cannot repair this incorrect order statement.

5.5 An infinite family

Proposition 5.1. *Let $0 \leq z < 1$, $s > 0$, $\alpha > 0$, $\beta > 0$, and let the exclusive cutoff satisfy $h \leq 0$. In the direct affine Lerch source path, the unchanged-atom-grade charging rule predicts a zero retained expression with order $\rho = s$ and bound*

$$\left(\frac{\alpha}{1 - z} + \beta\right) a^{-s}.$$

The actual source $\alpha\Phi(z, s, a) + \beta$ is not $\mathcal{O}(a^{-s})$.

Proof. Every exponent $s + k$ is strictly positive, so none is below h. The first omitted coefficient is $1/(1-z)$; at $N = 0$ the defining-sum majorant is $\alpha/(1-z)$. The scalar omission path leaves the positive atom grade unchanged. But $0 \le \Phi(z, s, a) \le a^{-s}/(1-z)$, so the actual source tends to the nonzero constant β. For sufficiently large a, the predicted bound is smaller than β, whereas the actual error is at least β. □

The proof concerns the rule's admitted mathematical regime. For special arguments such as $z = 0$, native simplification may eliminate the atom before the public dispatcher; such examples are not needed for the principal witness.

6 A correct remainder join

6.1 Common-grade theorem

Theorem 6.1. *Suppose $a \ge 1$ and an atom approximation $E(a)$ satisfies*

$$|\alpha\Phi(z, s, a) - E(a)| \le Ca^{-\rho}, \qquad C \ge 0.$$

If a constant $\beta \ne 0$ is omitted from the retained expression, then

$$|\alpha\Phi(z, s, a) + \beta - E(a)| \le (C + |\beta|)a^{-\widehat{\rho}}, \qquad \widehat{\rho} = \min(\rho, 0). \tag{6}$$

If the atom tail is exactly zero, use $\widehat{\rho} = 0$ and bound $|\beta|$. If $\beta = 0$, preserve the original tail unchanged.

Proof. The triangle inequality gives $Ca^{-\rho} + |\beta|$. If $\rho \ge 0$, then $a^{-\rho} \le 1$, so $C + |\beta|$ suffices. If $\rho < 0$, then $a^{-\rho} \ge 1$, so $|\beta| \le |\beta|a^{-\rho}$. These are precisely the two branches of (6). An exactly zero atom tail leaves only the constant; no multiplication involving an infinite exponent is necessary. □

For nonzero β and a genuinely decaying atom tail, exponent zero is also sharp in the sense of power order: the error tends to β, so a positive exponent cannot be valid. The conservative coefficient $C + |\beta|$ need not be sharp. Accuracy of the exponent and optimality of a bound constant are separate questions.

6.2 A sharper pointwise alternative

The intermediate bound

$$B_{\text{split}}(a) = Ca^{-\rho} + |\beta| \tag{7}$$

is often smaller than the common-grade bound. For the main witness it is $1 + 2/a^2$, rather than 3. The Python reference exposes both, but the supplied source candidate uses the simpler common-grade form so that the existing `RemainderBoundConstant` convention remains immediate.

Adopting (7) later would be a local strengthening of this repair, not an excuse to leave the asymptotic order at $\rho > 0$. A pointwise sum and a single power-order certificate must both describe the same full error.

6.3 Frontier metadata

The old atom frontier and the new full-source frontier can differ. When $\rho > 0$, the newly omitted constant is the leading error term. When $\rho = 0$, it combines with the atom's weight-zero frontier. When $\rho < 0$, it is lower order than the atom frontier. An exact atom plus an omitted constant has a nonzero error even though it has no omitted atom moment.

Accordingly, the candidate keeps `FirstOmittedMoment` as an atom index, updates `FrontierTerm` for the complete affine source, and adds `AtomRemainderPower` and `OmittedAffineConstant` only when a constant is actually charged. The complete result's `RemainderPower` comes from the joined grade. Cancellation of two weight-zero frontier coefficients may permit a sharper order, but the conservative order-zero result is sound without a new cancellation search.

7 The finite-source edge case

The same coding assumption is unsafe when the atom loop is exhausted. In that case `frontier===None` leads to $\rho = +\infty$, zero atom bound, and zero `RemainderScaleExpression`. A nonzero omitted constant must invalidate that exact-tail conclusion.

There is an algebraic example within the constructor's mathematical parameter grammar. Set $z = \sqrt{3}-2$ and $s = -3$. Its first four moments are

$$M_0 = \frac{1}{2} + \frac{\sqrt{3}}{6}, \quad M_1 = -\frac{1}{6}, \quad M_2 = -\frac{\sqrt{3}}{18}, \quad M_3 = 0.$$

Consequently,

$$\Phi(z, -3, a) = \left(\frac{1}{2} + \frac{\sqrt{3}}{6}\right) a^3 - \frac{1}{2}a^2 - \frac{\sqrt{3}}{6}a. \tag{8}$$

At cutoff zero all its nonzero rows have negative weight and are retained; the final constant moment vanishes. Adding an external constant one and then omitting that constant leaves an error exactly equal to one, not zero.

The identity was checked independently with SymPy. This review does *not* assert that the public Wolfram constructor reaches this exact private branch: symbolic evaluation may simplify a finite Lerch expression to a polynomial before dispatch. The artifact records that uncertainty. The case is included as a boundary condition for the private finishing rule and as a reason to handle $+\infty$ explicitly in the repair, rather than performing formal arithmetic with it.

8 Candidate source repair

8.1 Scope of the emitter

`code/emit_affine_lerch_patch.py` reads an existing checkout, verifies the pinned revision, isolates the text between `dirichletLerchForward` and the following dispatcher, checks five unique source anchors, and emits a unified diff outside the checkout. It does not modify a source file in place and does not rewrite the root standalone.

The changes add two local variables, preserve the original atom grade, replace the constant-charging rule, and update the affected metadata. No evaluator-wide symbol replacement, assumption change, special-function identity, term-goal policy, or coefficient recurrence is altered.

```
atomRho = rho;
(* frontierTerm is initialized from the atom before changing rho. *)
If[charged =!= 0,
  rho = If[atomRho === Infinity, 0, minOf[atomRho, 0]];
  boundConstant = boundConstant + Abs[charged];
  bound = boundConstant argument^(-rho);
  frontierTerm = Which[
    atomRho === Infinity || less[0, atomRho], charged,
    equal[atomRho, 0], frontierTerm + charged,
    True, frontierTerm]];
```

The existing private comparison helpers are used, avoiding a new dependency on a runtime-specific ordering routine. The infinite-tail branch is selected before a finite-grade comparison is required.

8.2 Invariants required after the change

A successful result must retain exactly the rows selected by the existing exclusive cutoff; the repair must not insert a constant at a forbidden weight. Its complete remainder grade must be no greater than the weight of any uncancelled omitted affine constant. An exact atom must remain distinguishable from an exact full affine result. The asymptotic object, its scalar bound, its frontier expression, and its provenance fields must describe one and the same error.

These are acceptance conditions for this concrete edit. They are not a claim that arbitrary raw result associations have been validated, nor a proposal to repeat the repository's broader result-schema work.

8.3 What has and has not been validated

Four synthetic emitter fixture methods pass: expected replacement, duplicate anchor refusal, changed anchor refusal, and refusal to apply the edit twice. Those tests establish defensive behavior of the emitter on its fixtures only. A complete checkout was unavailable, so the emitted candidate has not been applied to the actual modular distribution, rebuilt into the standalone, or loaded by either requested kernel. The source-level change should remain a candidate until those steps and the focused regressions succeed.

9 Independent tests and reproduction artifacts

9.1 An oracle independent of the faulty bound

For a positive integer s, rational $a > 0$, and rational $|z| < 1$, define

$$S_N = \sum_{n=0}^{N-1} \frac{z^n}{(a+n)^s}, \qquad R_N = \frac{|z|^N}{(1-|z|)(a+N)^s}.$$

The defining-sum tail has absolute value at most R_N, since every later $(a+n)^{-s}$ is bounded above by $(a+N)^{-s}$. For $z \geq 0$, a valid interval is $[S_N, S_N + R_N]$; for negative z, $[S_N - R_N, S_N + R_N]$ suffices. All these endpoints are exact rational numbers.

The test oracle compares the full affine error interval against the repaired bound. It does not compare the candidate to an identical copy of the candidate's proof rule. The moment routine is a reference

utility for existing coefficients, not a claimed new moment algorithm.

9.2 Executed populations

Population	Count	Result and scope
Independent Python test methods	20	All pass; rational model, oracle, counterexample, controls and input checks.
Concrete Lerch parameter combinations	3,000	All repaired bounds enclose the independent rational error intervals; these are subcases inside one method.
Abstract grade-and-bound combinations	725	The common-grade bound dominates the split bound; subcases inside one method.
Synthetic patch-emitter fixture methods	4	All pass; no actual checkout is patched.
Independent finite-source identity	1	Symbolic equality checked; not a kernel path observation.
Wolfram package executions	0	Unavailable.
Mathics package executions	0	Unavailable.

The Python method total is 24, not $24 + 3000 + 725$. The grids are deliberately reported separately. Logs, machine-readable counts, and counterexample data are included in `evidence/`. Decimal columns in the compact CSV are presentation values only; the actual checks use exact fractions.

9.3 Kernel probes and desired-contract tests

`ProbeAffineLerch.wl` is an unexecuted characterization script. It prints the runtime version, the relevant object fields at cutoffs $-1, 0, 1, 3$, and the exact comparison against $5/4$ at $x = 2$. It does not install a patch. `ReviewAffineLerch.wlt` contains eight unexecuted desired-contract tests for the common-grade repair, including negative multipliers, a zero-offset control, positive-cutoff controls, a shifted growing argument, and the two new provenance fields.

Run baseline characterization before changing the checkout. Then apply the candidate in a disposable copy, follow the repository's distribution-generation workflow, and run the focused tests on the canonical and rebuilt loading forms. The plain characterization script is distinct from MUnit infrastructure; Mathics support for a particular test runner must not be assumed merely because the mathematical inputs are portable.

10 Wolfram 15 and Mathics3 assessment for this finding

10.1 Shared mathematical exposure

The faulty constant-charging formula is in a shared canonical module. It uses no approximate numbers and has no Mathics-only conditional. The witness uses rational z, positive integer s, a positive real source approach, and an explicit package backend. Therefore the algebraic cause is common to both requested runtimes once this constructor is reached.[1]

This does not prove that all public evaluation steps are identical. Mathics can differ in preprocessing, simplification, parameter proofs, or module loading; such a difference could turn an anticipated wrong result into a refusal. A refusal would not vindicate the shared formula, but it would change the observed compatibility impact. That is why the report separates a source-proven local error from unrun public observations.

10.2 Repair portability

The candidate adds scalar arithmetic, branches, and calls to comparison helpers already used by the module. It introduces no new native special function, quantifier elimination, numerical precision requirement, or global System symbol modification. That makes it a small portability surface to validate, not an established portability result.

A focused acceptance run should check the exact retained expression, the joined order, the quantitative bound, and the frontier fields in each kernel. A positive-cutoff control must retain its original expression and atom order. Runtime-specific inability to reach a case must be recorded as a refusal or inconclusive test, not silently counted as a passing mathematical result.

11 Performance, API, and local development recommendations

11.1 Cost of the proposed repair

Once the atom frontier is known, the repair requires a constant number of scalar operations and order comparisons. It does not generate additional moments, expand new coefficients, enlarge a multi-index region, or add a numerical evaluation. “Constant number” here is an operation count, not a claim that arbitrary-precision symbolic comparisons take constant time. No Wolfram or Mathics timing improvement was measured.

The optional split bound in (7) is similarly cheap but changes the relationship between a pointwise bound expression and a normalized bound constant. It should be considered only after the common-grade correction is accepted, with explicit tests of both representations. There is no need to expand the scope into a general coefficient-algebra redesign to fix N01.

11.2 Make the surprising zero approximation interpretable

A zero approximation at cutoff zero is not itself a bug. It is a consequence of an exclusive cutoff applied to a function whose smallest nonzero weight is zero. The user-facing problem is falsely labeling its error as decaying. Documentation for this constructor should pair the two requests

$$h = 0 : \quad 0 + \mathcal{O}(1), \qquad h = 1 : \quad 1 + \mathcal{O}(a^{-2})$$

for the main witness, and explain that a sharper atom frontier cannot improve the grade of an omitted affine constant.

The added `OmittedAffineConstant` and `AtomRemainderPower` fields make that distinction inspectable without requiring the user to read an internal moment index. They are local explanatory metadata, not a substitute for the full-source remainder fields.

11.3 A bounded development sequence

First, capture both kernels' unmodified observations for the explicit rational witness and retain the negative-cutoff and positive-cutoff controls. Second, apply the minimal common-grade correction and validate the complete object contract, including the finite-tail finishing edge at the helper level. Third, add the paired cutoff example to the Lerch documentation and its focused regressions. Finally, evaluate the optional split pointwise bound only as a separate precision enhancement, with no change to the joined asymptotic grade.

This sequence is specific to the newly introduced affine-constant path. The existing review corpus remains the source for unrelated development priorities; this article does not reissue them under different names.

12 Conclusion

The main witness exposes a mistaken implication: a constant lies outside the requested retention cutoff, but it need not be dominated by the atom's first omitted scale. In a sparse expansion, the cutoff and the omitted grade can be far apart. Charging the constant's magnitude without joining its weight produces both a false explicit bound and a false big-$\mathcal{O}$ theorem.

The correction is small and mathematically elementary: preserve the atom tail, include the omitted constant as a weight-zero error, and assemble every full-source field from the resulting joined grade. The independent exact tests support that rule over the stated test domain; they do not establish package integration. The remaining decisive acceptance step is the supplied focused characterization and regression run in Wolfram 15 and Mathics3.

A Archive contents and commands

```
article/review.tex
article/review.pdf
code/affine_tail.py
code/test_affine_tail.py
code/emit_affine_lerch_patch.py
code/test_patch_emitter.py
code/ProbeAffineLerch.wl
code/ReviewAffineLerch.wlt
evidence/...
README.md
```

The exact independent tests use the Python standard library:

```
cd code
```

```
python -m unittest -v test_affine_tail test_patch_emitter
```

The optional finite-source identity used SymPy 1.14.0. To emit a candidate diff from a separately obtained checkout of the pinned revision:

```
python code/emit_affine_lerch_patch.py \
  --repo /path/to/Asymptotic \
  --output /path/outside/checkout/affine-lerch-candidate.patch
```

The emitter intentionally refuses a different revision. A newer implementation requires a source review and rebase, not bypassing the checks. The archive contains no checksum files, downloaded font files, upstream binaries, or purported kernel transcripts.

References

[1] Vladimir Reshetnikov, *Asymptotic*, `src/Kernel/DirichletSpecialFunctions.wl`, revision `8f280847bf8fd1f6488834cadf1542867292ce10`. See `dirichletSpecialAffineConstant`, `dirichletLerchForward`, and `dirichletSpecialMake`.

[2] Vladimir Reshetnikov, *Asymptotic*, `src/Kernel/AsymptoticAnalysis.wl`, same pinned revision; selected ranges described in the coverage record.

[3] *Code review reports*, retained-review index and retirement crosswalk, same revision.

[4] *Code review implementation status*, maintained register, same revision; selected sections used as the exclusion baseline.

[5] *Code review reports: wave 5*, retained findings and evidence boundaries, same revision.

[6] *Code review reports: wave 6*, retained findings and overlap table, same revision.

[7] *AsymptoticAnalysis: defining-sum integration, request semantics, and an integer-indexed Dirichlet algebra*, wave-6 report 55, article source, read at the current pin; the report's own analyzed snapshot is `8cee870994f506b501bae3ea6bd4a3a7edb895c1`. Relevant sections: affine lifting and the term-goal integration warning.

[8] *ReviewWave6Boundaries.wlt*, focused upstream tests, same pinned revision.

[9] NIST Digital Library of Mathematical Functions, §25.14, *Lerch's transcendent*, equation 25.14.1, `https://dlmf.nist.gov/25.14.E1`. Consulted 10 September 2026.

[10] Wolfram Research, *LerchPhi*, Wolfram Language documentation, `https://reference.wolfram.com/language/ref/LerchPhi.html`. Consulted 10 September 2026; only the positive-a, convergent defining-sum regime is used for the main proof.